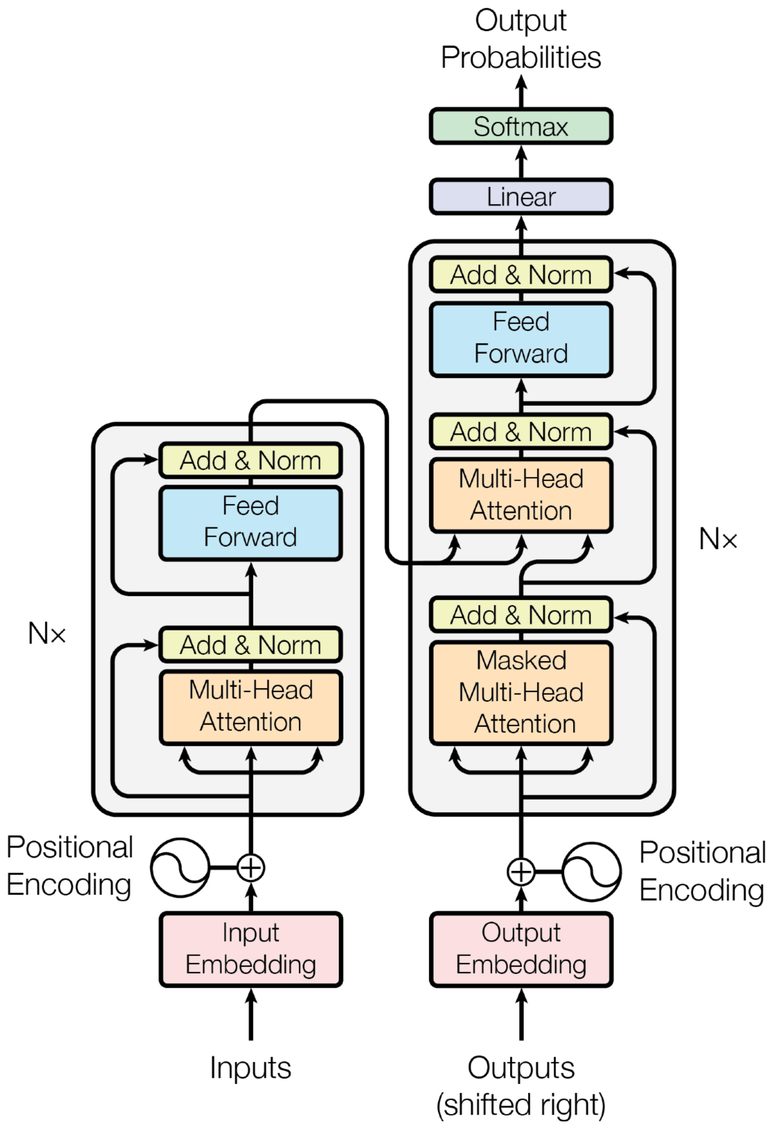
The rest of the Transformer

Antoine Bosselut

Full Transformer

- Made up of encoder and decoder
- Both encoder and decoder made up of multiple cascaded transformer blocks
  - slightly different architecture in encoder and decoder transformer blocks
- Blocks generally made up **multi-headed attention** layers (self-attention) and **feedforward** layers
- No recurrent computations!

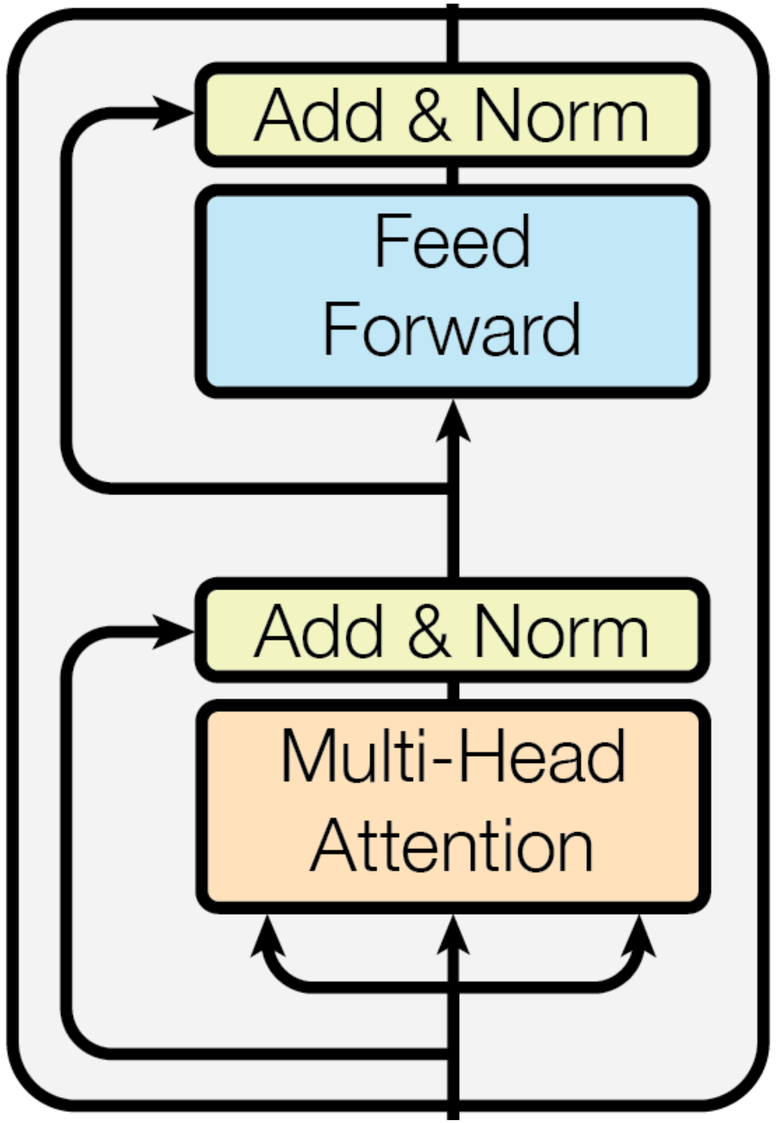
Encode sequences with self-attention



(Vaswani et al., 2017)

Transformer Block

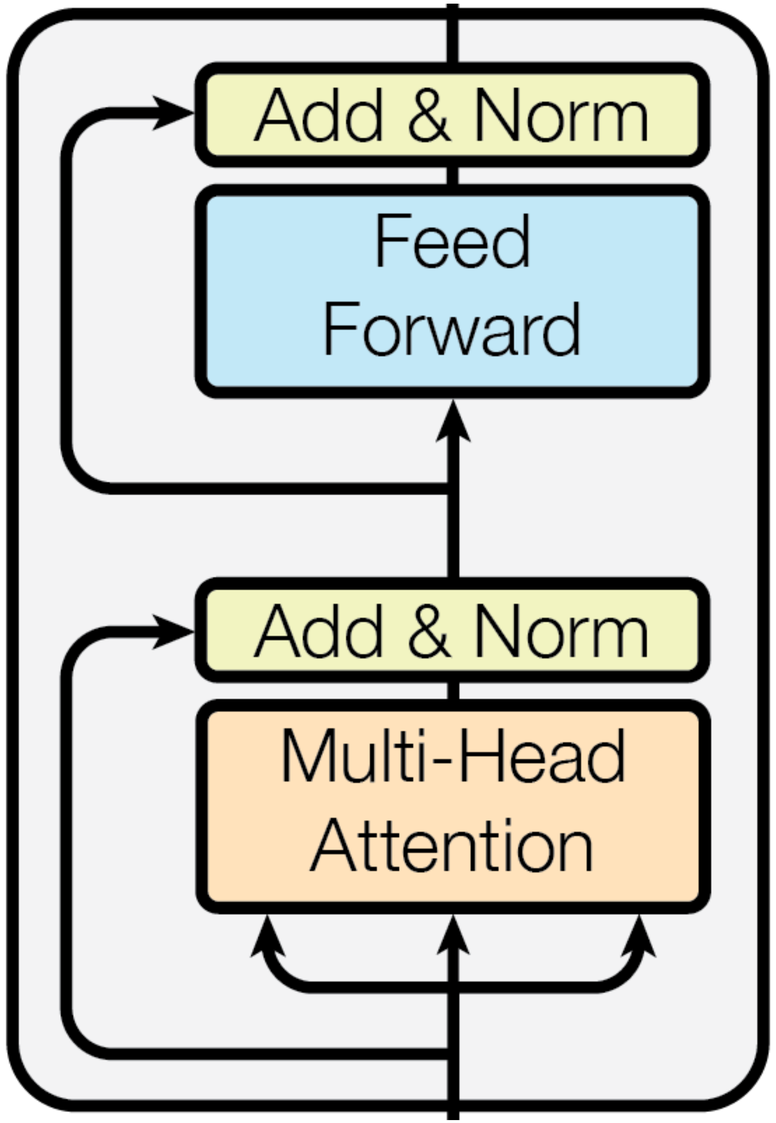
- Multi-headed attention is the main innovation of the transformer model!



Vaswani et al., 2017

Transformer Block

- Multi-headed attention is the main innovation of the transformer model!
- Each block also composed of:
  - a layer normalisations
  - a feedforward network
  - residual connections



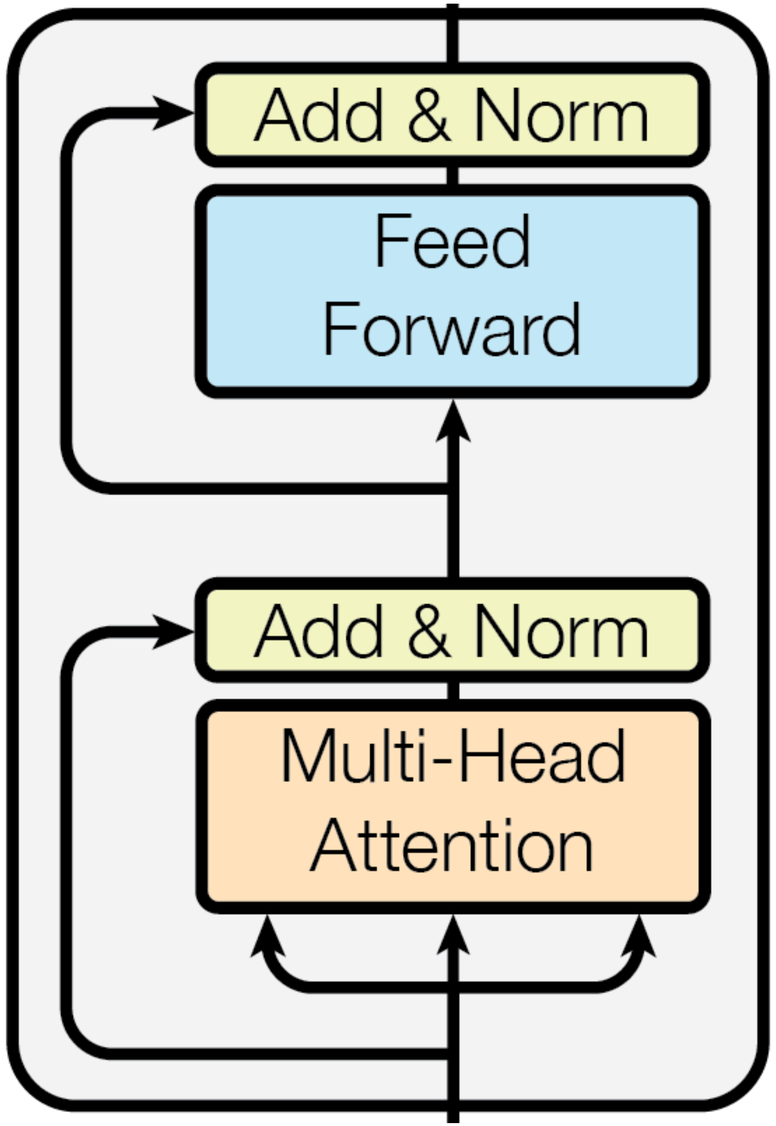
Vaswani et al., 2017

LayerNorm & Residual Connections

- **Layer Normalisation**
  - Normalize the outputs of different modules

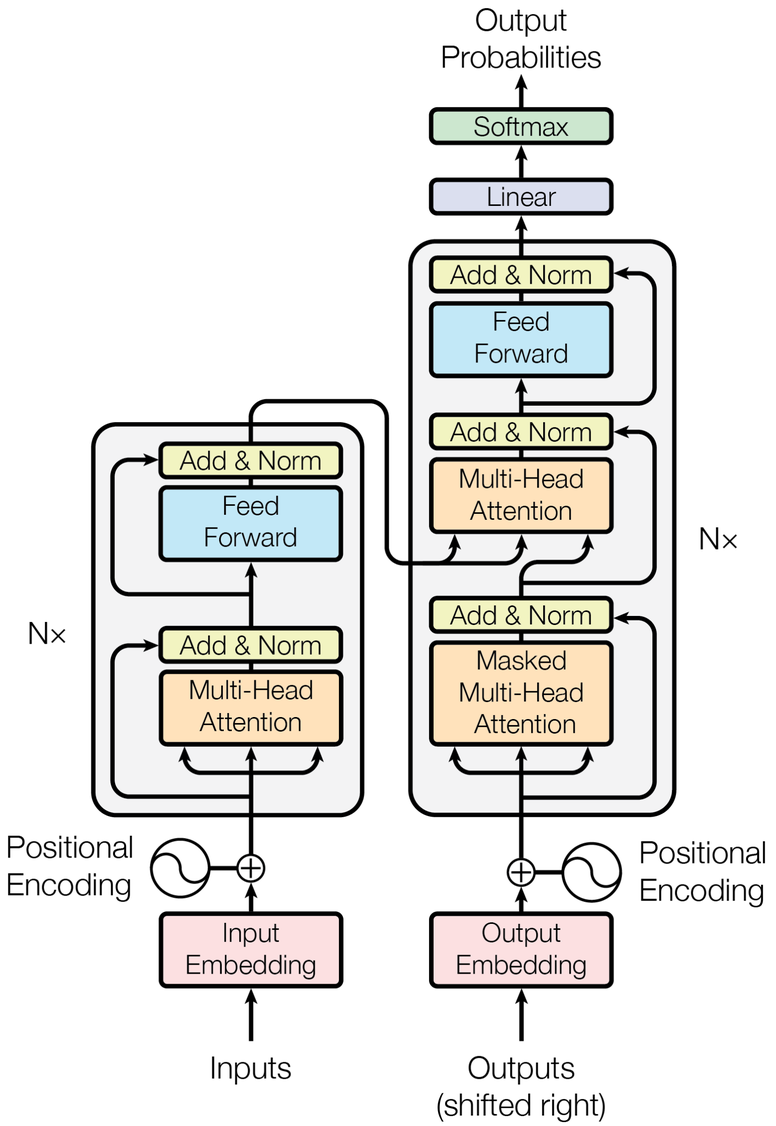
$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

- **Residual Connections**
  - Add the input of a module to its output
  - $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$



Full Transformer

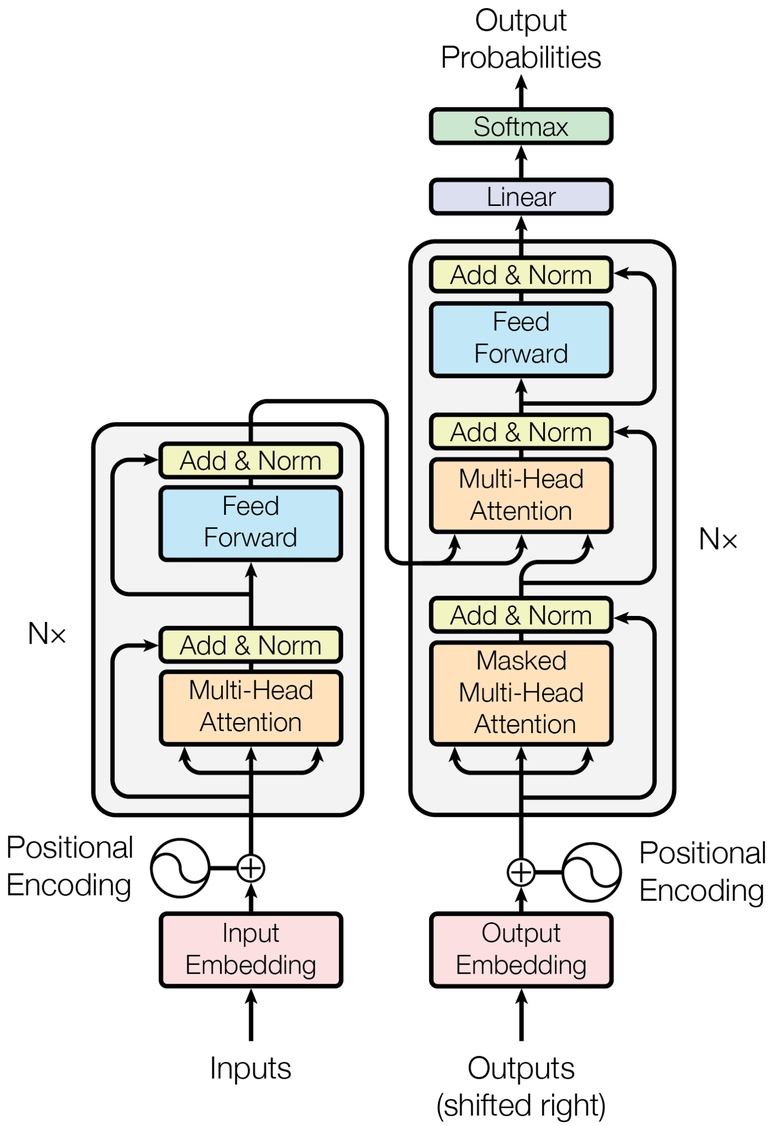
- Full transformer encoder is multiple cascaded transformer blocks
  - **build up compositional representations of inputs**



Vaswani et al., 2017

Full Transformer

- Full transformer encoder is multiple cascaded transformer blocks
  - **build up compositional representations of inputs**
- Transformer decoder (right) similar to encoder
  - First layer of block is **masked** multi-headed attention
  - Second layer is multi-headed attention over *final-layer* encoder outputs **(cross-attention)**
  - Third layer is feed-forward network



Vaswani et al., 2017

Question

What is an issue with self-attention for the decoder?

Masked Multi-headed Attention

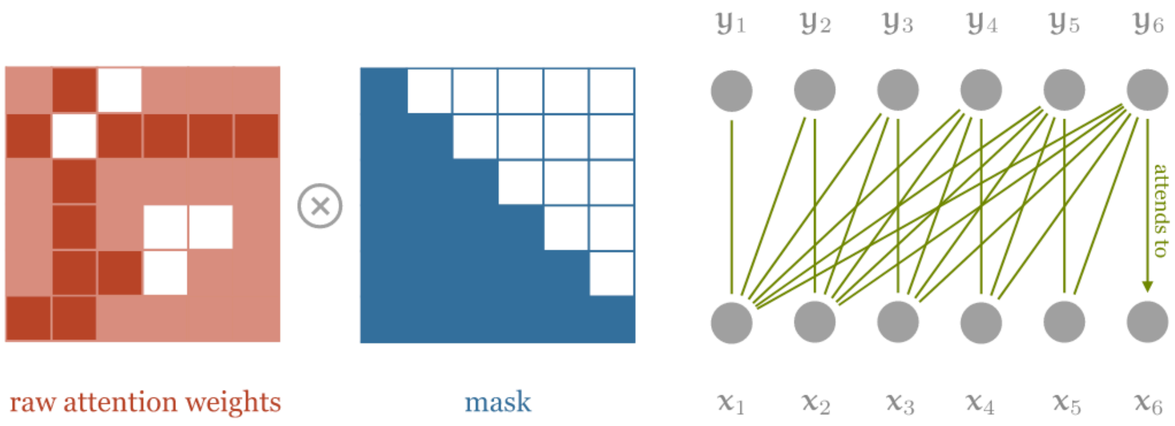
- Self-attention can attend to any token in the sequence
- For the decoder, **you don't want tokens to attend to future tokens**
  - Decoder used to generate text (i.e., machine translation)

Masked Multi-headed Attention

- Self-attention can attend to any token in the sequence

- For the decoder, **you don't want tokens to attend to future tokens**
  - Decoder used to generate text (i.e., machine translation)

Mask the attention scores of future tokens so their attention = 0



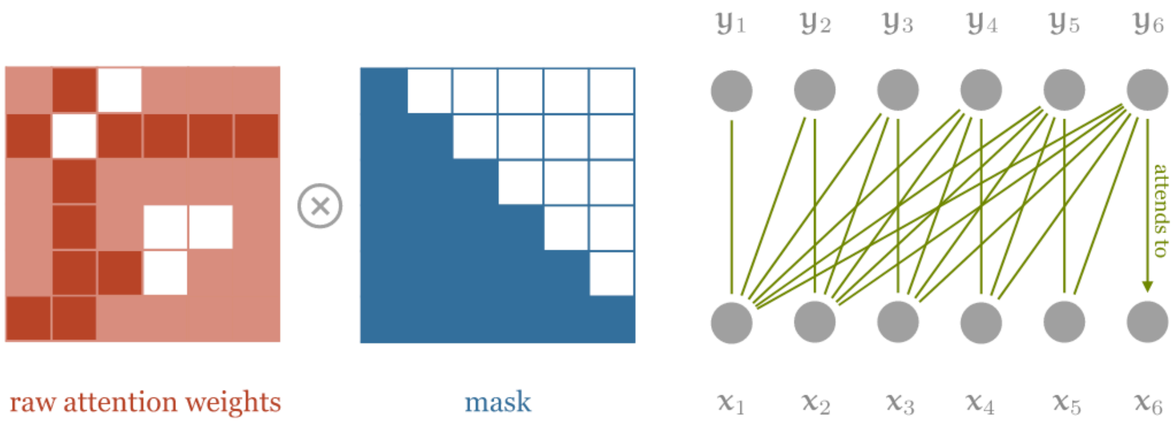
$$a_{st} = \frac{(\mathbf{W}^Q h_s^\ell)^T (\mathbf{W}^K h_t^\ell)}{\sqrt{d}} \quad \Rightarrow \quad a_{st} := a_{st} - \infty \; ; \; s < t$$

Masked Multi-headed Attention

- Self-attention can attend to any token in the sequence

- For the decoder, **you don't want tokens to attend to future tokens**
  - Decoder used to generate text (i.e., machine translation)

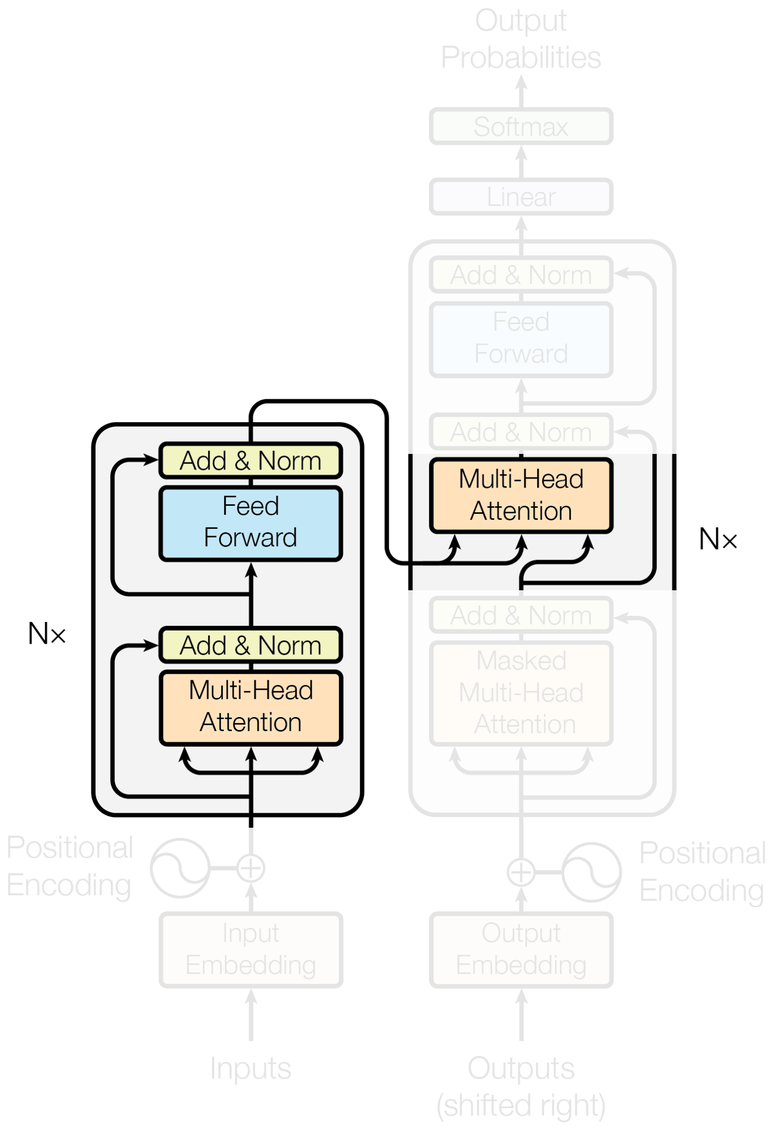
Mask the attention scores of future tokens so their attention = 0



$$a_{st} = \frac{(\mathbf{W}^Q h_s^\ell)^T (\mathbf{W}^K h_t^\ell)}{\sqrt{d}} \Rightarrow a_{st} := a_{st} - \infty \; ; \; s < t \Rightarrow \alpha_{st} = \frac{e^{a_{st}}}{\sum_j e^{a_{sj}}} = 0$$

Cross-attention

- **Cross attention** is the same classical attention as in the RNN encoder-decoder model
- Keys and values are output of **final** encoder block
- Query to the attention function is output of the masked multi-headed attention in the decoder
  - A representation from the decoder is used to **attend** to the encoder outputs



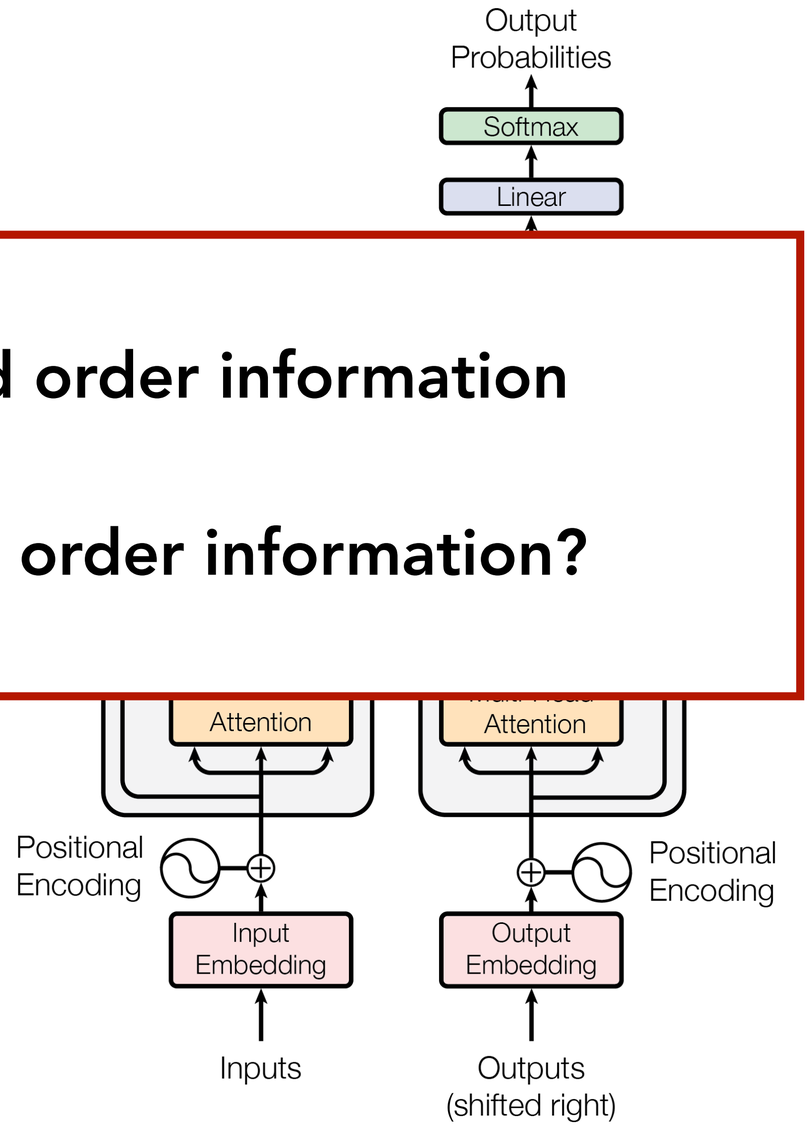
Vaswani et al., 2017

Full Transformer

- Full transformer encoder is multiple cascaded transformer blocks
  -
- Tr
  -
  - Second layer is multi-headed attention over encoder outputs **(cross-attention)**
  - Third layer is feed-forward network

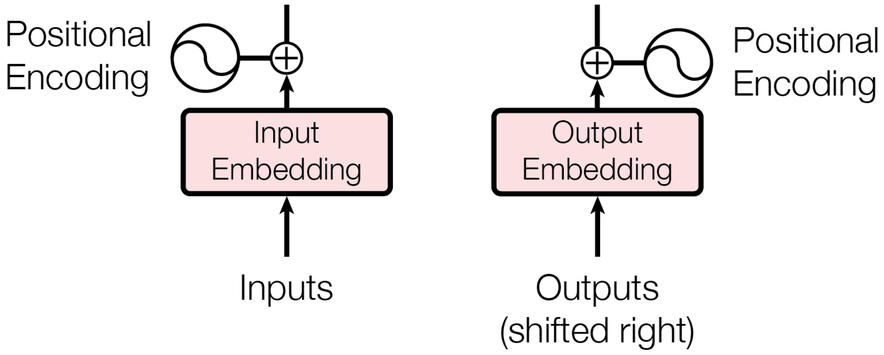
Recurrent models provided word order information

Does self-attention provide word order information?



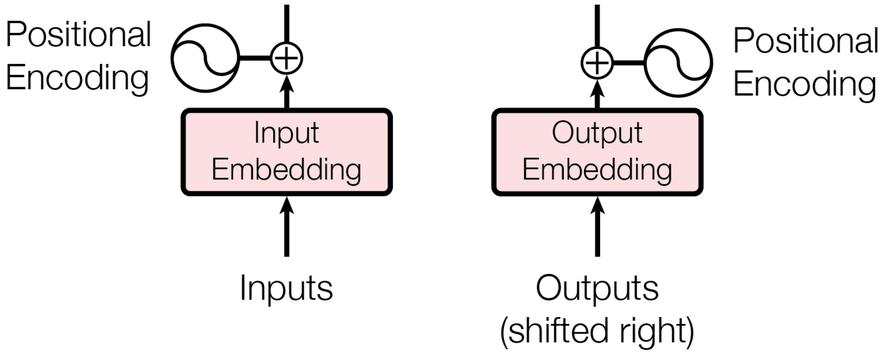
Vaswani et al., 2017

Position Embeddings



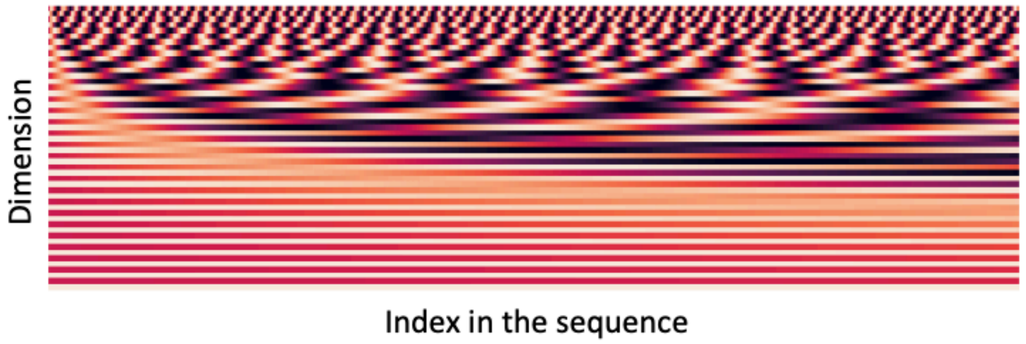
Vaswani et al., 2017

Position Embeddings



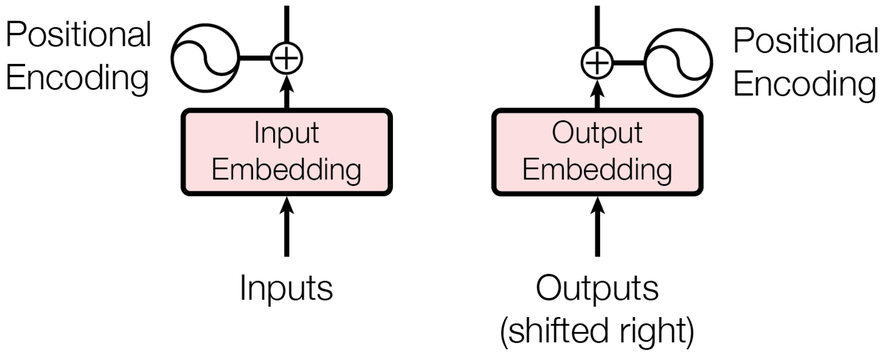
- Early position embeddings encoded a sinusoid function that was offset by a phase shift proportional to sequence position

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



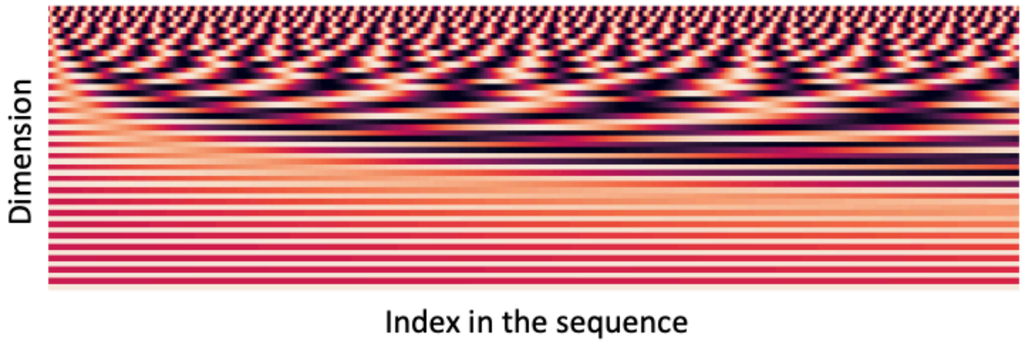
Vaswani et al., 2017

Position Embeddings



- Early position embeddings encoded a sinusoid function that was offset by a phase shift proportional to sequence position
- **In practice, easiest is to learn position embeddings from scratch**

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



Vaswani et al., 2017

Question

What might be a disadvantage of using learned position embeddings?

Poor generalisation to sequences longer than the maximum position embedding you have learned

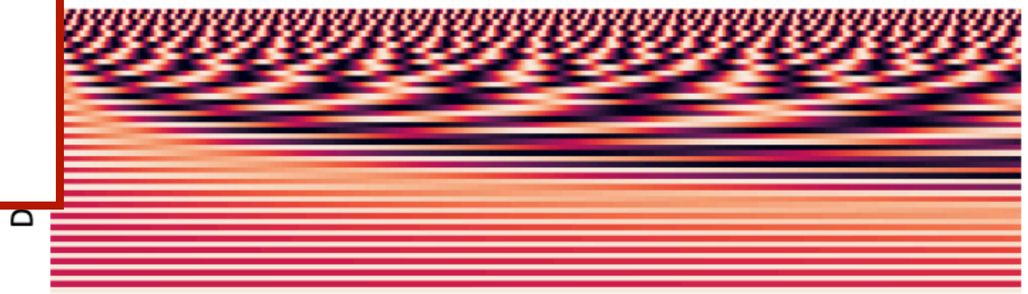
Position Embeddings

Lots of potential for new methods that generalise to longer sequences

Position embeddings remain an active area of research

- Early position embeddings encoded a sinusoid function that was offset by a phase shift proportional to sequence position
- **In practice, easiest is to learn position embeddings from scratch**

$$\begin{pmatrix} \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



Vaswani et al., 2017

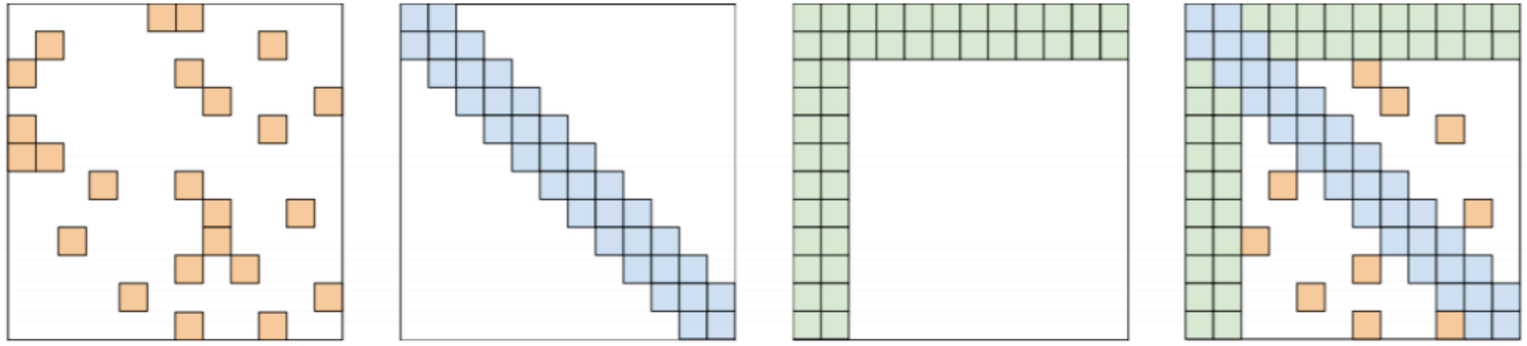
Performance: Machine Translation

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.0**	$2.3 \cdot 10^{19}$	

(Vaswani et al., 2017)

Question

What could be a disadvantage of transformers over RNNs?



(a) Random attention (b) Window attention (c) Global Attention (d) BIGBIRD

Other Resources of Interest

- The Annotated Transformer
  - https://nlp.seas.harvard.edu/2018/04/03/attention.html
- The Illustrated Transformer
  - https://jalammar.github.io/illustrated-transformer/
- Only basics presented here today! Many modifications to initial transformers exist

Recap

- **Temporal Bottleneck**: **Vanishing gradients** stop many RNN architectures from learning **long-range dependencies**
- **Parallelisation Bottleneck:** RNN states depend on previous time step hidden state, so must be **computed in series**
- **Attention**: Direct connections between output states and inputs (solves temporal bottleneck)
- **Self-Attention**: Remove recurrence, allowing parallel computation
- Modern **Transformers** use attention, but require position embeddings to capture sequence order

Tokenization

Antoine Bosselut

Today's Outline

- **Lecture**
  - **Tokenisation:** Controlling the input space

Some slides adapted from Danqi Chen, Greg Durrett

Tokenization

- The process of turning a stream of textual data into little pieces (tokens)
  - Words, terms, sentences, symbols, or some other meaningful elements

- **Token:** A sequence of characters that are grouped together as a useful semantic unit for processing

We all love the modern NLP course!

↓ **Tokenization**

```
["We", "all", "love", "the", "modern", "NLP", "course", "!"]
["We", "all", "love", "the modern", "NLP course", "!"]
["W", "e", "a", "l", "t", "h", "m", "o",…,"!"]
…
```

Why do we need tokenization?

Why do we need tokenization?

- Natural language text is a series unstructured sequences
- Tokenization breaks the text into chunks of information that can be considered as discrete elements
  - A numerical data structure suitable for machine learning

How should we tokenize text?

How to tokenize?

- Text can be analyzed and generated at many granularities
  - From bytes to multi-word expressions
- Traditionally, NLP models operated over words
  - Splits the data using space and delimiters (e.g., “ ” or “;” or “,”)
  - Still complex!
- So far in the course, we’ve assumed the text was cut into words

Word Tokenization

- It takes natural breaks, like pauses in speech or spaces in the text
  - Splits the data into its respective words using delimiters (e.g., “ ” or “;” or “,”)

We all love the modern NLP course!

↓ **Tokenization**

`[“We”, “all”, “love”, “the”, “modern”, “NLP”, “course”, “!”]`

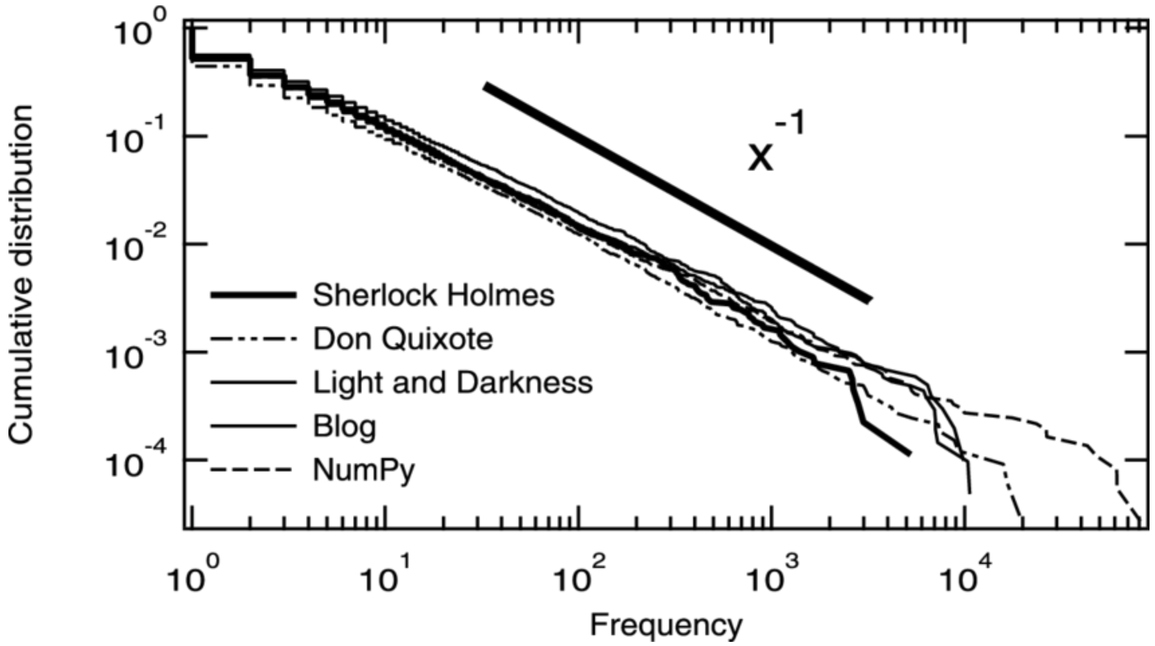
Word Tokenization: Challenges

- Not as simple as splitting on whitespace and punctuation
  - Ambiguity of the word boundaries
  - Example: Prof. Bosselut / Dr. Who / 3-year-old / don't / 123,456.78
- It requires many specialized rules to handle specific inputs
  - Examples: spaCy and NLTK tokenizers
- What about languages & scripts without spaces?:
  - Chinese, Japanese, Thai, Urdu, traditional Korean, Hindi, Tamil, and others

Word Tokenization: Challenges

- It treats different forms of the same word as separate types
  - (e.g., “talk”, “talks”, “talked”, “talking”, etc)
  - **Problematic when training over smaller datasets**

- Leads to a big vocabulary
  - Zipf’s Law
  - A huge embedding matrix for the input and the output layers
  - Require more computational resources



Word Tokenization: Challenges

- How to deal with **out-of-vocabulary (OOV)** words?
  - No way of assigning an index to an unseen word
  - No word embedding for that word and cannot process the input sequence
- Replace low-frequency words in training data with a special **<UNK>** token
  - Use this token to handle unseen words at test time too
  - We lose lots of information about texts with a lot of rare words/entities

What else could we do?

Character Tokenization

- Split the raw text into individual characters

We all love the modern NLP course!

↓ **Tokenization**

```
["W", "e", "a", "l", "l", "l", "o", "v", "e", "t", "h", "e"
 "m", "o", "d", "e", "r" "n", "N", "L", "P",
 "c", "o", "u", "r", "s", "e", "!"]
```

Character Tokenization

- Small vocabulary:
  - The **number of unique characters** in the training data
- Solves the OOV words problem
- More robust to noise and out-of-distribution data

Character Tokenization

- Small vocabulary:
  - The **number of unique characters** in the training data
- Solves the OOV words problem
- More robust to noise and out-of-distribution data
- length of the input increases rapidly
  - **Slower** training and inference
  - Challenging to learn the relationship between the characters to form meaningful words

What else could we do?

Subword Tokenization

- A solution between word and character-based tokenization
  - Splits the text into subwords (or n-gram characters)

We all love the modern NLP course!

↓ **Tokenization**

```
["We", "al", "l", "lo", "ve", "the", "modern", "NL", "P", "course", "!"]
```

Subword Tokenization

- Uses the following principles:
  - Frequently used words should not be split into smaller subwords
  - Rare words should be decomposed into meaningful subwords
- These methods have two parts:
  - A token learner that takes a raw training corpus and induces a vocabulary (a set of tokens)
  - A token segmenter that takes a raw test sentence and tokenizes it according to that vocabulary

Subword Tokenization

- More meaningful individual tokens
- Manageable vocabulary size
- Treats different forms of the same word similarity
- Reduces the impact of the OOV words problem:
  - Segments OOV as subwords and represents the word in terms of these subwords

Byte Pair Encoding (BPE)

- BPE was initially a data compression algorithm:
  - Find the best way to represent data by identifying the common byte pairs
- Used by OpenAI for tokenization when pretraining the original GPT model
- Widely used tokenization method among transformer-based LMs
  - GPT, GPT-2, RoBERTa, BART, DeBERTa, Llama etc.
- Represent the entire text dataset with the least amount of tokens

(Rico Sennrich et al., 2016)

BPE - Training Algorithm

- The training process applies the following steps:
  1. Split the words into characters (after appending </w> to words to identify boundaries)
  2. Create the base vocabulary from the unique characters in the corpus:
  3. Compute the frequency of a pair of characters
  4. Merge the most common character pairing
  5. Add this to the list of tokens and recalculate the frequency count for each token
  6. Rinse and repeat steps **3 to 5** until you have reached your **defined token limit** or a **set number of iterations**

(Rico Sennrich et al., 2016)

BPE - Example

- Suppose we are given the following corpus:

Corpus		
newest	lower	low
newest	lower	low
newest	widest	low
newest	widest	low
newest	widest	low

BPE - Example

1. Split the words into characters:
   - Append the **end of the word** (e.g., </w>) symbol to every word in the corpus:

Corpus		
n e w e s t </w>	l o w e r </w>	l o w </w>
n e w e s t </w>	l o w e r </w>	l o w </w>
n e w e s t </w>	w i d e s t </w>	l o w </w>
n e w e s t </w>	w i d e s t </w>	l o w </w>
n e w e s t </w>	w i d e s t </w>	l o w </w>

BPE - Example

2. Create the **base vocabulary** from the unique characters in the corpus:

Frequency	
d	3
e	15
i	3
l	7
n	5
o	7
s	8
t	5
w	15
</w>	15

Vocabulary									
d	e	i	l	n	o	s	t	w	</w>

BPE - Example

Iteration 1: Merge the most common character pairing

1. Compute character pair frequencies:

Frequency	
(d, e): 3	(l, o): 7
(e, r): 2	(n, e): 5
(e, s): 8	(o, w): 7
(e, w): 5	(r, </w>): 2
(i, d): 3	(s, t): 8
(t, </w>): 8	(w, </w>): 5
(w, e): 7	(w, i): 3

BPE - Example

Iteration 1: Merge the most common character pairing

1. Compute character pair frequencies:
2. Merge the most frequent pair

Frequency	
(d, e): 3	(l, o): 7
(e, r): 2	(n, e): 5
(e, s): 8	(o, w): 7
(e, w): 5	(r, </w>): 2
(i, d): 3	(s, t): 8
(t, </w>): 8	(w, </w>): 5
(w, e): 7	(w, i): 3

Corpus		
n e w **es** t </w>	l o w e r </w>	l o w </w>
n e w **es** t </w>	l o w e r </w>	l o w </w>
n e w **es** t </w>	w i d **es** t </w>	l o w </w>
n e w **es** t </w>	w i d **es** t </w>	l o w </w>
n e w **es** t </w>	w i d **es** t </w>	l o w </w>

BPE - Example

Iteration 1: Merge the most common character pairing

1. Compute character pair frequencies
2. Merge the most frequent pair
3. Add them to the vocabulary

Vocabulary									
d	e	i	l	n	o	s	t	w	</w>
es									

BPE - Example

Iteration 2: Merge the most common character pairing

1. Compute character pair frequencies

Frequency	
(d, es): 3	(l, o): 7
(e, r): 2	(n, e): 5
(es, t): 8	(o, w): 7
(e, w): 5	(r, </w>): 2
(i, d): 3	(w, es): 5
(t, </w>): 8	(w, </w>): 5
(w, e): 2	(w, i): 3

BPE - Example

Iteration 2: Merge the most common character pairing

1. Compute character pair frequencies
2. Merge the most frequent pair

Frequency	
(d, es): 3	(l, o): 7
(e, r): 2	(n, e): 5
(es, t): 8	(o, w): 7
(e, w): 5	(r, </w>): 2
(i, d): 3	(w, es): 5
(t, </w>): 8	(w, </w>): 5
(w, e): 2	(w, i): 3

Corpus		
n e w **est** </w>	l o w e r </w>	l o w </w>
n e w **est** </w>	l o w e r </w>	l o w </w>
n e w **est** </w>	w i d **est** </w>	l o w </w>
n e w **est** </w>	w i d **est** </w>	l o w </w>
n e w **est** </w>	w i d **est** </w>	l o w </w>

BPE - Example

Iteration 2: Merge the most common character pairing

1. Compute character pair frequencies
2. Merge the most frequent pair
3. Add them to the vocabulary

Vocabulary									
d	e	i	l	n	o	s	t	w	</w>
es	**est**								

BPE - Example

Iteration 3: Merge the most common character pairing

1. Compute character pair frequencies
2. Merge the most frequent pair

Frequency	
(d, est): 3	(l, o): 7
(e, r): 2	(n, e): 5
(est, </w>): 8	(o, w): 7
(e, w): 5	(r, </w>): 2
(i, d): 3	(w, est): 5
(w, e): 2	(w, </w>): 5
	(w, i): 3

Corpus		
n e w **est</w>**	l o w e r </w>	l o w </w>
n e w **est</w>**	l o w e r </w>	l o w </w>
n e w **est</w>**	w i d **est</w>**	l o w </w>
n e w **est</w>**	w i d **est</w>**	l o w </w>
n e w **est</w>**	w i d **est</w>**	l o w </w>

BPE - Example

Iteration 3: Merge the most common character pairing

1. Compute character pair frequencies
2. Merge the most frequent pair
3. Add them to the vocabulary

Vocabulary									
d	e	i	l	n	o	s	t	w	</w>
es	est	**est</w>**							

BPE - Example

- After 10 iterations the vocabulary is:

Vocabulary									
d	e	i	l	n	o	s	t	w	</w>
es	est	est</w>	lo	low	low</w>	ne	new	newest</w>	

BPE - Training Algorithm

- The training process applies the following steps:
  1. Split the words into characters (after appending </w>)
  2. Create the base vocabulary from the unique characters in the corpus:
  3. Compute the frequency of a pair of characters
  4. Merge the most common character pairing
  5. Add this to the list of tokens and recalculate the frequency count for each token
  6. Rinse and repeat **3 to 5** steps until you have reached your **defined token limit** or a **set number of iterations**

(Rico Sennrich et al., 2016)

Note: Pre-tokenization

- BPE assumes some existing splitting of raw text into “word-like” units before running
- **Pre-tokenization** produces a rough initial segmentation that the tokeniser subsequently refines
- **Common pre-tokenisation operations**: whitespace & punctuation splitting, unicode normalisation, special character stripping / handling
- Languages without whitespace require **dedicated segmentation tools** instead of simple whitespace splitting

What's a shortcoming of BPE?

Always merge greedily based on raw frequency

Other Subword Tokenization

- WordPiece (Yonghui Wu et al., 2016):
  - Merge the pairs based not only on frequency, but also how frequent elements are individually

$$\text{score} = (\text{freq_of_pair})/(\text{freq_of_first_element} \times \text{freq_of_second_element})$$

- SentencePiece (Taku Kudo et al., 2018):
  - Train subword models directly from raw sentences (i.e., no pre-tokenisation)
  - Encode everything as Unicode (including spaces)

How many subwords do we need?

Number of Subwords

- Considerations:
  - Number of tasks
  - Number of domains
  - Number of languages and scripts
- The smaller the dataset on which you train, the smaller the subword vocabulary should be
- With lots of data, larger vocabulary is more computationally intensive at training, but potentially more efficient at inference time

Limitations of Subword Tokenization

- Subwords do not necessarily correspond to morphemes
  - Not optimal for agglutinative languages (e.g., Turkish) and non-concatenative morphology (e.g., Arabic)

كتب	k-t-b	"write" (root form)
كَتَبَ	**k**a**t**a**b**a	"he wrote"
كَتَّبَ	**k**a**tt**a**b**a	"he made (someone) write"
اِكْتَتَبَ	i**k**ta**t**a**b**a	"he signed up"

Table 1: Non-concatenative morphology in Arabic. When conjugating, letters are interleaved *within* the root. The root is therefore not separable from its inflection via any contiguous split.

(Jonathan H. Clark et al., 2022)

Limitations of Subword Tokenization

- Pretokenization rules do not work in some languages:
  - **Example**: Thai and Chinese do not use spaces between words
  - **Example:** Hawaiian and Twi use punctuation as consonants

- Struggling with challenging domains:
  - Informal text including typos, spelling variation, transliteration, or emoji

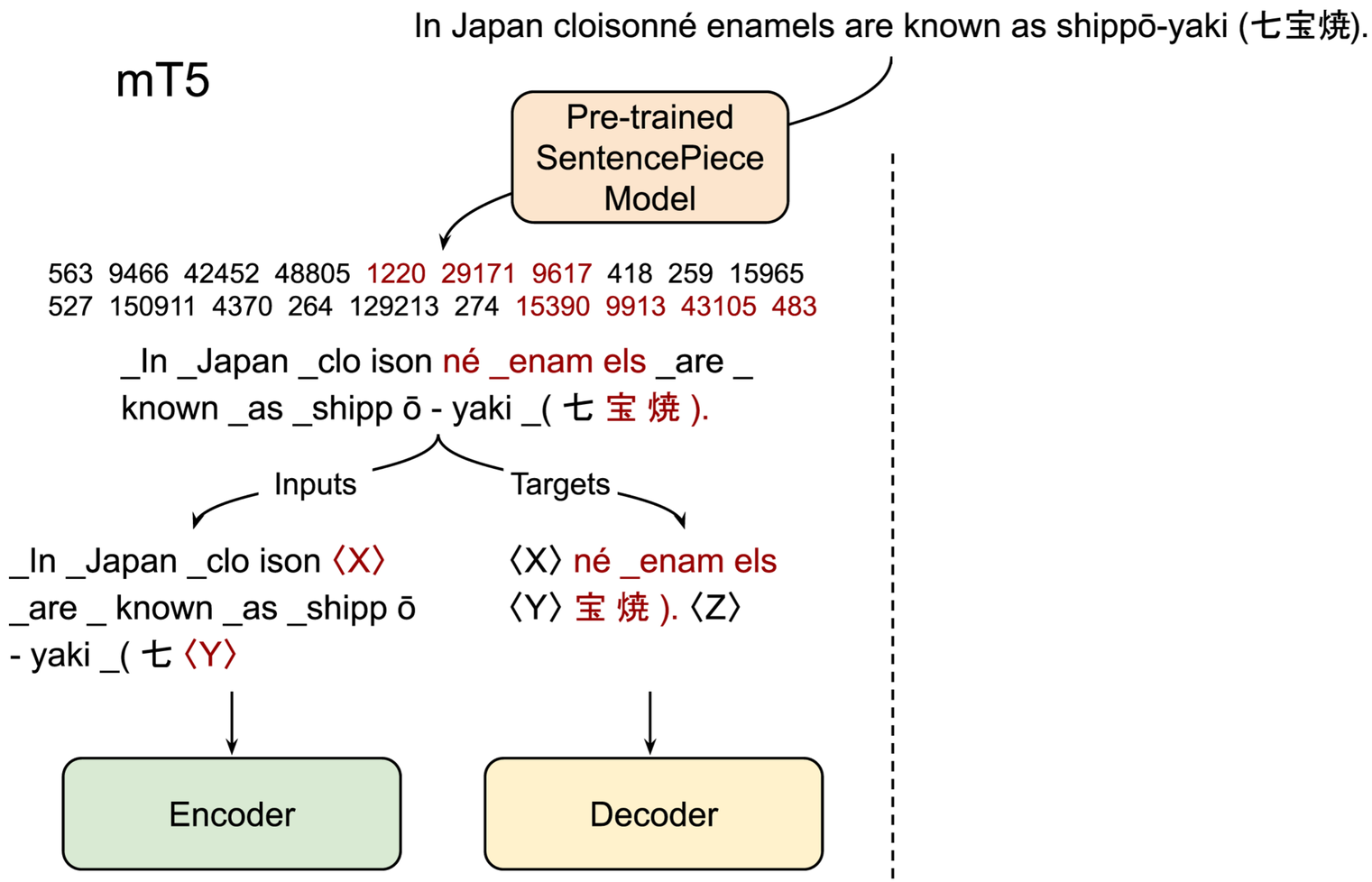
Byte-level Tokenization

- Represent text using the byte sequence resulting from a standard encoding like UTF-8 (e.g., ByT5)
  - Avoiding the huge-vocabulary problem of character-level models
- More robust to noise and out-of-distribution data
- Avoidance of out-of-vocabulary issues

Byte-level Tokenization

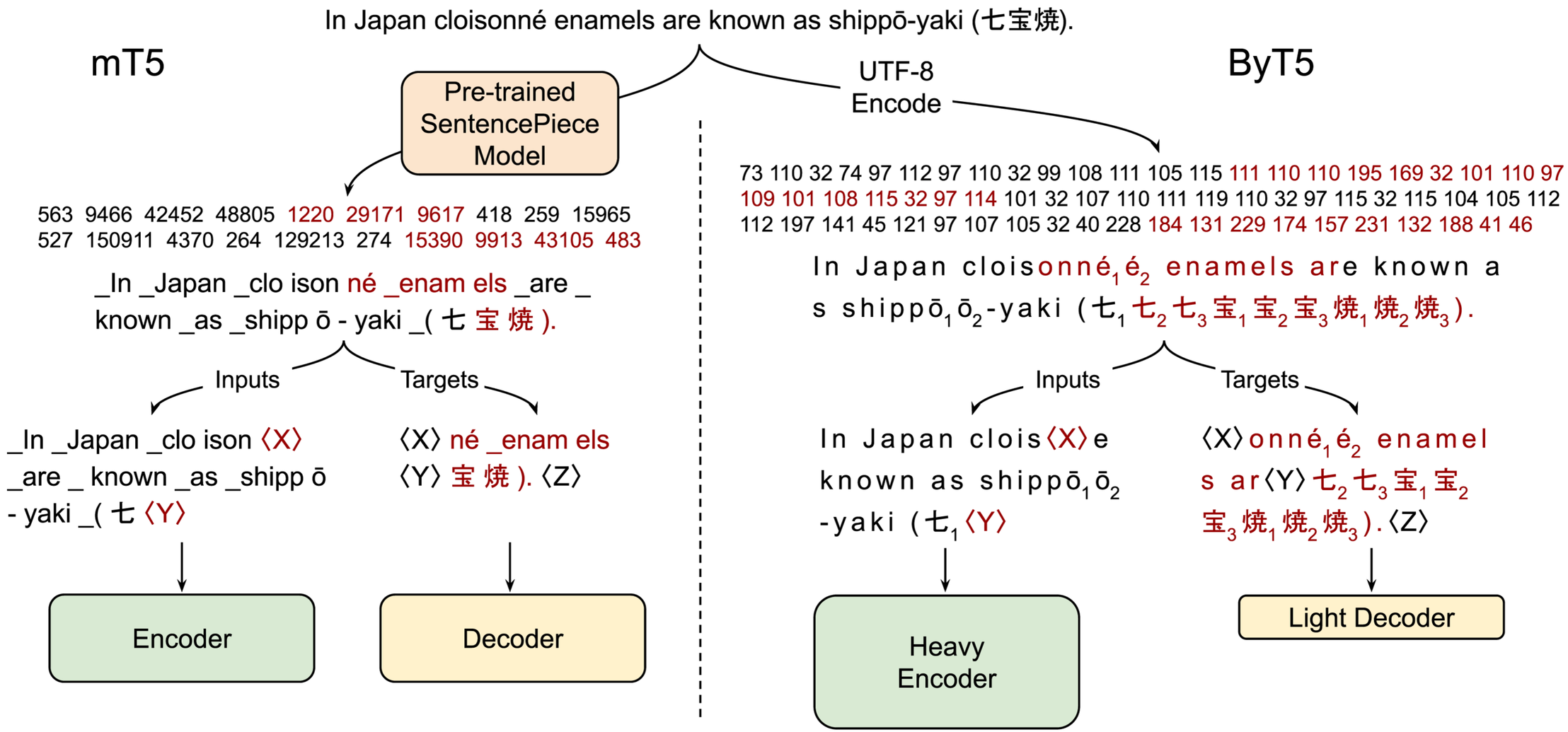
- Represent text using the byte sequence resulting from a standard encoding like UTF-8 (e.g., ByT5)
  - Avoiding the huge-vocabulary problem of character-level models
- More robust to noise and out-of-distribution data
- Avoidance of out-of-vocabulary issues
- **Longer training and inference time due to longer sequences**

ByT5: Tokenizer Free



(Linting Xue et al., 2022)

ByT5: Tokenizer Free



(Linting Xue et al., 2022)

ByT5: Tokenizer Free

- Fewer parameters associated with vocabulary

Size	Params	mT5 Vocab	ByT5 Vocab
Small	300M	85%	0.3%
Base	582M	66%	0.1%
Large	1.23B	42%	0.06%
XL	3.74B	27%	0.04%
XXL	12.9B	16%	0.02%

- Improvement in tasks with noisy data

- Dealing with increased sequence length:
  - Training with shorter sequences (max 1024 bytes)

(Linting Xue et al., 2022)

References

- Mielke, Sabrina J., et al. "Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP." arXiv preprint arXiv:2112.10508 (2021).

- Xue, Linting, et al. "Byt5: Towards a token-free future with pre-trained byte-to-byte models." Transactions of the Association for Computational Linguistics 10 (2022): 291-306.

- Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural machine translation of rare words with subword units." arXiv preprint arXiv:1508.07909 (2015).